

Kofeko AI Resume Evaluation

Simple guide: parsing, weights, skills, years, and how the match % works · June 2026

30-second summary: Kofeko reads your resume file → sends text + job description + skill weights to AI (Replicate) → AI returns a **0–100% match score** and a detailed breakdown. There is no fixed calculator formula in code; the AI follows written rules. **Weight (0–10)** tells the AI which skills matter most for your job.

1. Two ways a resume gets “parsed”

Flow	When	Purpose
Profile parse	Candidate uploads on My Profile	Auto-fill skills, experience, education
Evaluation parse	Evaluate with AI / AI Lab	Score resume against a specific job

Step A — Extract text (not AI)

- **PDF** → pdf-parse library
- **DOCX** → mammoth library
- **TXT** → read directly
- Max size: **8 MB**
- Evaluation uses first **~14,000 characters** of resume text

Step B — AI structures & scores

The AI extracts summary, skills, jobs, education, projects, hobbies — then compares everything to your job.

2. What goes into an evaluation?

1. **Job title** (e.g. Senior MERN Developer)
2. **Job description (JD)** — full role text
3. **Skill priorities** — each skill with **weight 0–10**
4. **Resume text** from the file

Important about “years”: When you set *years of experience* on a skill in JD Creator / job posting, that value is **saved on the job** but is **not sent to the evaluation AI today**. Only **skill name + weight** are sent. The AI still reads how many years the candidate actually has from resume dates and job history.

3. Skill weight — what it means

Weight = how important this skill is for YOUR company on THIS job.

Weight	Typical meaning
9–10	Must-have / deal-breaker
7–8	Very important
5–6	Important
3–4	Nice to have
1–2	Minor / bonus

For each priority skill the AI returns:

Field	Meaning
matched	true if resume shows evidence (React = React.js OK)
contribution	How much this skill helped the score (strong evidence + high weight = more)
evidence	Short quote from resume

- Skill **missing** → matched false, contribution 0
- Skill **found** → matched true, contribution scales with weight + evidence strength
- **No weights set?** AI infers priorities only from JD text (less predictable)

4. Weight vs years — what's the difference?

	Weight (0–10)	Years on skill (job setting)
Meaning	How much you <i>care</i> about the skill	How many years you <i>expect</i>

Used in evaluation?	Yes	Not yet (AI reads years from resume)
Affects score how?	Missing high-weight skill hurts a lot	Via experience section if resume shows short/long tenure

Example A — same skills, different weights

Job: Full Stack Developer · React weight **10**, Python weight **3**

- **Candidate A:** 3 years React on resume → likely **higher** score
- **Candidate B:** Expert Python, no React → loses on weight-10 skill → **lower** score

Example B — same skill, different years on resume

Both have React (weight 10). Candidate X: 5 years React. Candidate Y: 6 months React.

- Both can show **matched: true** for React
- Candidate X usually gets higher **experience** section score → higher **overall %**
- AI also considers seniority fit in **roleFitNotes**

Example C — two jobs, same candidate

Same resume. Job 1 (React Lead): React weight 10. Job 2 (Java Backend): React weight 2, Java weight 10.

The **same skills** produce **different scores** because weights change what matters.

5. The match % (0–100)

Overall score = the main number you see (e.g. 78% match).

Section scores (each 0–100)

Section	What it measures
skills	Alignment with required/preferred skills
experience	Relevant roles, tenure, career path
projects	Portfolio work vs job needs
education	Degrees and relevance

professionalSummary	Summary / headline fit
hobbies	Almost no impact on overall (by design)

What pushes score UP

- High-weight skills matched with clear evidence
- Experience in the same domain and seniority as JD
- Strong projects using the job's tech stack
- Clear career progression

What pushes score DOWN

- Missing must-have skills (weight 8–10)
- Wrong specialty (e.g. ML resume for frontend role)
- Weak or unrelated experience
- Listed in **matchScoreBreakdown.deductions** (factor + points + reason)

6. Full flow

```

Resume file (PDF / DOCX / TXT)
  ↓
Extract plain text
  ↓
Job title + JD + skill weights (0–10 each)
  ↓
AI (Replicate) – hiring analyst
  ↓
Results:
  • Overall % (0–100)
  • Section scores
  • Per-skill: matched, contribution, evidence
  • Strengths, gaps, risks, interview recommendation
  ↓
Saved as Evaluation (production) or shown in AI Lab (test only)

```

7. Where to test

Feature	Where
---------	-------

Generate JD + suggested skills	JD Creator
Score one candidate	Job posting → applicant → Evaluate with AI
Score many resumes (no DB save)	Sidebar → AI Lab
Parse profile only	Candidate My Profile

8. Quick reference

Question	Answer
What is weight?	Skill importance 0–10 for this job
What is years on skill?	Expected years (stored; not in AI prompt yet)
Is score a formula?	No — AI applies rules, not a spreadsheet
AI provider?	Replicate (REPLICATE_API_TOKEN)

Kofeko · Based on analyzeResume.ts, parseResume.ts, evaluation.service.ts · June 2026